FROTA-SENAI

Guia de implantação e operação

Docker, runbook, backup, configuração e troubleshooting

Versão documental 1.0

26 de outubro de 2026

Uso interno — SENAI

Documento derivado dos arquivos Markdown oficiais em docs/. Não substitui o código-fonte.

Sumário

Os títulos abaixo correspondem às seções deste PDF. A fonte editável está em docs/. Diagramas Mermaid não são desenhados aqui.

Guia de implantação — FROTA-SENAI

- Requisitos (produção Compose)
- Preparar o servidor (recomendado, não automatizado neste repo)
- Subir
- Pós-implantação
- Atualização
- Rollback (recomendado)
- Desenvolvimento versus produção
- Falhas no boot

Runbook operacional — FROTA-SENAI

1. Estado dos serviços
2. Logs
3. Frontend não carrega
4. API 502 / 401 inesperado
5. Banco
6. Reinício seguro
7. Disco
8. Rotina operacional parada
9. Escalação
10. Confirmar recuperação

Backup e restauração

- O que proteger
- PostgreSQL (Compose)
- Mídia
- Frequência, retenção, local (propostos)
- Verificação
- Storage Supabase

Configuração e variáveis de ambiente

- Arquivos
- Variáveis (Django / Compose)
- JWT (código, não env)
- Cuidados

Troubleshooting

Guia de implantação — FROTA-SENAI

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno
Procedimento Docker	Extraído de `compose.yaml`, Dockerfiles, `README-DOCKER.md` e `docker-entrypoint.sh`
TLS no Compose	Não implementado (recomendado à frente do Nginx)

Complementa README-DOCKER.md.

Requisitos (produção Compose)

- Debian com Docker Engine 24+ e `docker compose`.
- Porta `HTTP_PORT` livre (padrão 80).
- Rede de saída no build (PyPI, npm). Em runtime, FIPE e Google Fonts são opcionais.
- Disco para volumes `pgdata`, `media`, `staticfiles`.

Preparar o servidor (recomendado, não automatizado neste repo)

1. Usuário no grupo `docker`.
2. Diretório da aplicação (exemplo genérico: `/opt/frota` — ajuste à política local).
3. Copiar o projeto sem `.env` de outro ambiente.
4. `[ -f .env ] || cp .env.example .env` e preencher chaves.

Subir

```
docker compose up -d --build
docker compose ps
docker compose exec backend python manage.py createsuperuser
```

Ordem: `db` healthy → `backend` (espera PG, `migrate`, `collectstatic`, Gunicorn) → `web`. `rotina` sobe com `SKIP_MIGRATE=1` após db e backend healthy.

Implementado: este fluxo está no Compose e no entrypoint.

Não verificado nesta revisão: execução real em servidor de produção.

Pós-implantação

```
curl -sI "http://127.0.0.1:${HTTP_PORT:-80}/" | head -5
curl -sI "http://127.0.0.1:${HTTP_PORT:-80}/admin/login/" | head -5
curl -s -o /dev/null -w "%{http_code}\n" -X POST \
  -H "Content-Type: application/json" -d '{}' \
  "http://127.0.0.1:${HTTP_PORT:-80}/api/usuarios/login/"
docker compose exec backend python manage.py check
docker compose exec backend python manage.py showmigrations | tail -20
```

O login JWT é POST (`CustomTokenObtainPairView`). Esperado com corpo vazio: 400. GET no mesmo URL: 405. Credenciais inválidas em POST: 401. 502 = proxy/backend. Esta checagem não valida TLS.

Acessos: `/` SPA, `/api/`, `/admin/`, `/media/`.

Atualização

```
# preserve .env
docker compose up -d --build
```

O backend reaplica migrate. Backup antes: backup-restauracao.md.

Rollback (recomendado)

Não há ferramenta de rollback no repositório. Procedimento proposto (pendente de ensaio):

1. Manter artefato/código da versão anterior.
2. Restaurar dump PostgreSQL e tarball de mídia testados.
3. `docker compose up -d --build` na versão anterior.
4. Não usar `down -v`.

Desenvolvimento versus produção

	Dev	Compose (prod documentado)
DEBUG	True (default settings)	False
API	:8000 direto	Nginx `/api`
SPA	Vite :5173	estáticos no Nginx
DB	settings ou Supabase no example do BACKEND	`db:5432`, SSL disable
Rotina	comando manual	serviço `rotina`
CORS LAN	ligado se DEBUG	`CORS_ALLOW_LAN=False`

Falhas no boot

Sintoma	Causa provável
backend unhealthy	migrate lento, `ALLOWED_HOSTS` sem localhost, Gunicorn não sobe
502	backend ainda não escuta
DisallowedHost	host público ausente em `ALLOWED_HOSTS`
CSRF Admin	origem sem esquema em `CSRF_TRUSTED_ORIGINS`

Runbook operacional — FROTA-SENAI

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno
Risco	Não usar `docker compose down -v` nem `flush`

Pré-condição: diretório do Compose e permissão Docker.

1. Estado dos serviços

```
docker compose ps
docker compose exec backend python manage.py check
```

Esperado: `db`, `backend`, `rotina`, `web` Up (backend/web healthy).

2. Logs

```
docker compose logs -f --tail=100
docker compose logs -f backend
docker compose logs -f rotina
docker compose logs -f web
docker compose logs -f db
```

Loggers de aplicação: `reservas.operacoes`, `vistorias.desempenho` (stdout).

3. Frontend não carrega

1. `curl -sI http://127.0.0.1:${HTTP_PORT:-80}/`
2. Logs `web`.
3. Confirmar build da imagem (SPA está na imagem, não em volume de código).

4. API 502 / 401 inesperado

1. Logs `backend`.
2. Healthcheck: `docker compose exec backend python -c "import urllib.request; urllib.request.urlopen(urllib.request.Request('http://127.0.0.1:8000/admin/login/', headers={'Host':'localhost'}), timeout=5)"`
3. 401 em rotas autenticadas é normal sem Bearer.

5. Banco

```
docker compose exec db pg_isready -U frota -d frota
```

Ajuste usuário/banco ao `.env` (`DB_USER`, `DB_NAME`).

Não publique a porta 5432 no host (Compose atual não publica).

6. Reinício seguro

```
docker compose stop
docker compose start
# ou
docker compose up -d
```

Reinício do `backend` dispara migrate de novo (idempotente se já aplicado).

7. Disco

```
df -h
docker system df
```

Volumes críticos: `pgdata`, `media`. Limpeza `docker system prune` não é rotina (risco de apagar imagens ainda úteis; não remove volumes nomeados sem `-a --volumes`).

8. Rotina operacional parada

Sintoma: reservas aprovadas sem check-in não cancelam após a meia-noite do dia seguinte.

```
docker compose logs rotina --tail=50
docker compose exec backend python manage.py rotina_operacional
```

O comando avulso no backend executa uma passagem. O serviço `rotina` é o loop.

9. Escalação

1. Reproduzir horário (fuso America/Sao_Paulo).
2. Guardar `docker compose logs` (sem colar `.env`).
3. Verificar auditoria (`/auditoria`) se a ação deveria ter gerado registro.
4. Envolver TI para SMTP, DNS, TLS e disco.

10. Confirmar recuperação

- `docker compose ps` healthy
- Login na SPA
- `POST /api/usuarios/login/` com `{}` responde 400 (GET nesse URL responde 405)

- Uma reserva de teste em homologação, nunca dados de produção improvisados

Backup e restauração

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno
Status dos procedimentos	Documentados no README-DOCKER; restauração não foi ensaiada nesta revisão

O que proteger

Ativo	Onde	Criticidade
Banco PostgreSQL	volume `pgdata`	máxima
Uploads	volume `media`	máxima (CNH, termos, fotos)
`.env`	host	máxima (segredos) — backup offline controlado, nunca no Git
Código	repositório	alta
`staticfiles`	regenerável via collectstatic	baixa
Cadeia de auditoria	tabela no banco	máxima; depende da chave HMAC

PostgreSQL (Compose)

Substitua `frota` se `DB_USER`/`DB_NAME` forem outros.

```
docker compose exec -T db pg_dump -U frota -d frota -Fc > "frota-$(date +%Y%m%d).dump"
```

Restauração (destrutiva para o conteúdo atual do banco, não para o volume Docker):

```
docker compose exec -T db pg_restore -U frota -d frota --clean --if-exists < frota-AAAAMMDD.dump
```

A opção `--clean --if-exists` recria os objetos dentro do banco. Não use `docker compose down -v`: a flag `-v` apaga os volumes `pgdata` e `media`.

Não execute restauração em produção sem janela e backup imediatamente anterior. Este procedimento não foi ensaiado nesta revisão. Integridade do dump só é válida após restore de teste em ambiente isolado.

Mídia

```
docker compose exec -T backend tar czf - -C /app/media . > "media-$(date +%Y%m%d).tar.gz"
docker compose exec -T backend tar xzf - -C /app/media < media-AAAAMMDD.tar.gz
```

A extração sobrescreve arquivos no volume `media`; não remove o volume. Ensaio apenas em clone, nunca nos volumes de produção.

Frequência, retenção, local (propostos)

Não há job de backup no Compose. Metas recomendadas (não contratadas no código):

Item	Proposta
Frequência	diário do banco; mídia diária ou contínua se o volume permitir
Retenção	14–30 dias de dumps; 1 cópia offsite
Local	armazenamento institucional criptografado, fora do mesmo disco do servidor
RPO proposto	24 h (se backup diário)
RTO proposto	a definir pela TI após ensaio

Verificação

1. Restore em clone Docker sem volumes de produção.
2. `manage.py check`, login, abrir um arquivo `/media/` conhecido.
3. `GET /api/auditoria/integridade/` (admin) — deve permanecer íntegra se a chave HMAC for a mesma.

Storage Supabase

Se `USE_SUPABASE_STORAGE=True`, o tarball de `/app/media` não cobre os objetos do bucket. Backup do bucket é responsabilidade da plataforma — não documentado como procedimento testado aqui.

Configuração e variáveis de ambiente

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno
Fontes	`.env.example` (raiz), `BACKEND/.env.example`, `SISTEMA-FINAL/.env.example`, `config/settings.py`, `compose.yaml`

Não reproduza valores de `.env` reais. Placeholders abaixo são fictícios.

Arquivos

Arquivo	Uso
`FROTA/.env`	Compose (não versionado)
`FROTA/.env.example`	Modelo Docker Debian
`BACKEND/.env`	`runserver` local / outros hosts
`BACKEND/.env.example`	Modelo dev com comentários Supabase/Render
`SISTEMA-FINAL/.env`	opcional `VITE_API_URL` no Vite

O Compose define `DB_HOST=db`, `DB_PORT=5432` e, por padrão, `DEBUG=False`, `USE_SUPABASE_STORAGE=False`.

Variáveis (Django / Compose)

Nome	Obrigatória em produção	Padrão no código	Quem usa	Exemplo seguro
`SECRET_KEY`	Sim	`django-insecure-change-me`	Django	`troque-por-token-urlsafe-50`
`AUDITORIA_CHAVE_INTEGRIDADE`	Fortemente recomendada	igual a `SECRET_KEY`	HMAC auditoria	chave estável distinta
`DEBUG`	`False`	`True` no settings se omitido	Django	`False`

Nome	Obrigatória em produção	Padrão no código	Quem usa	Exemplo seguro
ALLOWED_HOSTS	Sim	localhost	Django; healthcheck precisa de localhost	localhost,127.0.0.1,frota.exemplo.local
CSRF_TRUSTED_ORIGINS	Admin Django	vazio	CSRF	http://frota.exemplo.local
CORS_ALLOWED_ORIGINS	Sim se SPA em outra origem	localhost:3000/5173	corsheaders	mesma origem pública
CORS_ALLOW_LAN	Não	igual a DEBUG	regex RFC1918 :3000/:5173	False em produção
USE_X_FORWARDED_HOST	Compose: True	False	URLs atrás do proxy	True
SECURE_PROXY_SSL_HEADER	Se TLS no proxy	False	X-Forwarded-Proto	True só com HTTPS
DB_ENGINE	Sim	postgresql	Django	django.db.backends.postgresql
DB_NAME / DB_USER / DB_PASSWORD	Sim	name postgres, user vazio	Django, entrypoint	frota / senha forte
DB_HOST / DB_PORT	Sim	host vazio, porta 6543	Django	Compose: db / 5432
DB_SSLMODE	Conforme rede	require	psycopg	Compose interno: disable
DB_DISABLE_SERVER_SIDE_CURSORS	Pooler	True	Django	Compose: False
USE_SUPABASE_STORAGE	Não	False	STORAGES	False no Compose mínimo
SUPABASE_S3_*	Se storage True	—	django-storages	não usar no Compose mínimo
MEDIA_ROOT / STATIC_ROOT	Compose	BACKEND/media, staticfiles	Django	/app/media, /app/staticfiles
FROTA_EMAIL_ENABLED	Não	False	notificacoes.py	False até a TI autorizar
EMAIL_BACKEND	Se e-mail	dummy	Django	django.core.mail.backends.smtp.EmailBackend
EMAIL_HOST EMAIL_PORT EMAIL_HOST_USER EMAIL_HOST_PASSWORD EMAIL_USE_TLS EMAIL_USE_SSL DEFAULT_FROM_EMAIL	Se e-mail	ver settings	Django	placeholders; nunca senha real na documentação
VEICULO_IMAGEM_TAMANHO_MAXIMO_MB	Não	5	veículos	5
VEICULO_IMAGEM_MAXIMO_FOTOS	Não	8	veículos	8
SKIP_MIGRATE / SKIP_COLLECTSTATIC	Só rotina	unset	entrypoint	1 no serviço rotina
HTTP_PORT	Não	80	Compose	80 ou 8080
VITE_API_URL	Build frontend	detecção automática	Vite	Compose: /api

JWT (código, não env)

ACCESS_TOKEN_LIFETIME: 8 h. REFRESH_TOKEN_LIFETIME: 7 dias. Rotação e blacklist: True.

Cuidados

- Trocar AUDITORIA_CHAVE_INTEGRIDADE após dados existentes faz a tela de integridade falhar.
- DEBUG=True força ALLOWED_HOSTS=['*'].
- Não commitar .env. O .gitignore da raiz já ignora.

Troubleshooting

Campo	Valor
Versão	1.0
Data	26 de outubro de 2026
Classificação	Uso interno

Sintoma	Causas possíveis	Diagnóstico	Solução	Validação
502 Bad Gateway	backend fora, migrate em andamento	`docker compose logs backend`	aguardar healthy; `docker compose up -d`	`curl -X POST /api/usuarios/login/` com `{}` → 400
DisallowedHost	host ausente	log Django	incluir host em `ALLOWED_HOSTS` sem remover localhost	healthcheck + acesso pelo hostname
CSRF 403 no Admin	origem não listada	console da aba Admin	`CSRF_TRUSTED_ORIGINS` com `http(s)://`	salvar um objeto no Admin
SPA chama :8000 em produção	imagem web buildada sem `VITE_API_URL=/api`	rede do browser	rebuild com ARG do Compose	requests em `/api/` same-origin
Check-in recusado no PC	`exigir_cliente_mobile`	mensagem da API	usar celular	POST registrar 201
Check-in fora de horário	06:00–22:00 ou ±30 min	horário do servidor `America/Sao_Paulo`	esperar janela ou antecipação	201 na janela
Check-out unidade errada	`unidade_checkin` diferente	reserva na API	mesma unidade do check-in	201
Reserva não cancela sozinha	`rotina` parada	`docker compose logs rotina`	subir serviço; comando avulso	status `cancelada` após meia-noite aplicável
FIPE 500	Parallelum indisponível	log traceback	cadastrar veículo manualmente	POST veículo 201
Integridade auditoria falha	chave HMAC alterada ou DB adulterado	`GET /auditoria/integridade/`	não “consertar” hashes; restaurar backup e chave correta	`integra: true`
`/media/` 404	volume vazio ou upload antes do mount	`docker compose exec backend ls /app/media`	conferir volume; reenviar arquivo	URL 200
Login 400 sempre	senha ou modo 1º acesso invertidos	corpo da resposta	seguir fluxo de `precisa_definir_senha`	200 + tokens
Funcionário vê tela admin	`is_admin=True` no cadastro	perfil JSON	corrigir flag (procedimento de RH/TI, não automático)	rotas adminOnly redirecionam
E-mail não chega	`FROTA_EMAIL_ENABLED=false`	log `reservas.operacoes` “desligado”	não ligar sem SMTP institucional	log “enviado” só se habilitado
permission denied Docker	usuário fora do grupo	`groups`	`usermod -aG docker` + nova sessão	`docker compose ps`
Disco cheio	dumps, logs, media	`df -h`	arquivar dumps; não apagar pgdata	serviços healthy

Comandos destrutivos (`down -v`, `pg_restore --clean` em produção, `flush`) não são solução rotineira.